```
!pip install transformers torch torchvision requests pandas matplotlib
-q
```

```
import pandas as pd

base =
"https://raw.githubusercontent.com/NationalGalleryOfArt/opendata/main/
data"

print("Downloading NGA metadata...")
objects = pd.read_csv(f"{base}/objects.csv", low_memory=False)
images  = pd.read_csv(f"{base}/published_images.csv",
low_memory=False)

objects.columns = objects.columns.str.lower()
images.columns  = images.columns.str.lower()

print(f"Total artworks:  {len(objects):,}")
print(f"Published images: {len(images):,}")
print(f"\nObject columns: {list(objects.columns[:8])}")
print(f"Image columns:  {list(images.columns[:8])}")
```

```
Downloading NGA metadata...
Total artworks:  144,192
Published images: 130,938

Object columns: ['objectid', 'uuid', 'accessioned', 'accessionnum',
'locationid', 'title', 'displaydate', 'beginyear']
Image columns:  ['uuid', 'iiifurl', 'iiifthumburl', 'viewtype',
'sequence', 'width', 'height', 'maxpixels']
```

```
# Filter to paintings only
paintings = objects[
    objects["medium"].str.lower().str.contains(
        "oil|tempera|watercolor", na=False
    )
].copy()

print(f"Total artworks:     {len(objects):,}")
print(f"Paintings only:     {len(paintings):,}")
print(f"\nSample paintings:")
print(paintings[["title", "medium",
"displaydate"]].head(8).to_string(index=False))
```

```
Total artworks:     144,192
Paintings only:     23,112

Sample paintings:
```

```
title                    medium         displaydate

Saint James Major        tempera on panel             c. 1310
                                                                Saint Paul
and a Group of Worshippers        tempera on panel               1333
                          Saint Andrew and Saint Benedict with the
Archangel Gabriel [left panel] tempera on poplar panel shortly before
1387
      Madonna and Child Enthroned with Twelve Angels, and with the
Blessing Christ [middle panel] tempera on poplar panel shortly before
1387
Saint Bernard and Saint Catherine of Alexandria with the Virgin of the
Annunciation [right panel] tempera on poplar panel shortly before 1387

The Madonna of Humility        tempera on panel             c. 1430
                                                                     Saint
Anthony Abbot [left panel]        tempera on panel                1354
                   Madonna and Child Enthroned with Four Saints and
Eighteen Angels [middle panel]        tempera on panel
1354
```

Dataset: National gallery of art 23,112 paintings filtered from 144,192 total artworks as the dataset also contained sculptures etc so we have only taken paintings. Filtered using medium keywords (oil. tempera, watercolor). Sample shows many early renaissance religious works suggesting NGA collection to be eighted toward older european art. Unline wikiart, there are no style/ genre labels so similarity must be found purely from visual features

```
# Join paintings with image URLs
merged = paintings.merge(images[["uuid","iiifthumburl"]],
                         on="uuid", how="inner")
merged = merged.dropna(subset=["iiifthumburl"])

print(f"Paintings with images: {len(merged):,}")
print(f"\nSample image URL:")
print(merged["iiifthumburl"].iloc[0])
```

```
Paintings with images: 0

Sample image URL:

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_569/3175574074.py in <cell line: 0>()
      6 print(f"Paintings with images: {len(merged):,}")
      7 print(f"\nSample image URL:")
----> 8 print(merged["iiifthumburl"].iloc[0])

/usr/local/lib/python3.12/dist-packages/pandas/core/indexing.py in
```

```
__getitem__(self, key)
   1189             maybe_callable = com.apply_if_callable(key,
self.obj)
   1190             maybe_callable =
self._check_deprecated_callable_usage(key, maybe_callable)
-> 1191             return self._getitem_axis(maybe_callable,
axis=axis)
   1192
   1193     def _is_scalar_access(self, key: tuple):

/usr/local/lib/python3.12/dist-packages/pandas/core/indexing.py in
_getitem_axis(self, key, axis)
   1750
   1751             # validate the location
-> 1752             self._validate_integer(key, axis)
   1753
   1754             return self.obj._ixs(key, axis=axis)

/usr/local/lib/python3.12/dist-packages/pandas/core/indexing.py in
_validate_integer(self, key, axis)
   1683         len_axis = len(self.obj._get_axis(axis))
   1684         if key >= len_axis or key < -len_axis:
-> 1685             raise IndexError("single positional indexer is
out-of-bounds")
   1686
   1687     #
---------------------------------------------------------------------

IndexError: single positional indexer is out-of-bounds
```

```python
print("Objects uuid sample:")
print(objects["uuid"].head(3).tolist())

print("\nImages uuid sample:")
print(images["uuid"].head(3).tolist())

print("\nObjects columns:", list(objects.columns))
print("Images columns:", list(images.columns))
```

```
Objects uuid sample:
['b5dc500c-be86-4b05-93fa-59a25d8775fb', '59e0e07c-208e-4d24-a3d3-
f4a56d20d6d3', '95cfbfd3-5947-4834-b1cf-bf92ee1a7340']

Images uuid sample:
['00007f61-4922-417b-8f27-893ea328206c', '0000bd8c-39de-4453-b55d-
5e28a9beed38', '0001668a-dd1c-48e8-9267-b6d1697d43c8']

Objects columns: ['objectid', 'uuid', 'accessioned', 'accessionnum',
'locationid', 'title', 'displaydate', 'beginyear', 'endyear',
'visualbrowsertimespan', 'medium', 'dimensions', 'inscription',
```

```
'markings', 'attributioninverted', 'attribution', 'provenancetext',
'creditline', 'classification', 'subclassification',
'visualbrowserclassification', 'parentid', 'isvirtual',
'departmentabbr', 'portfolio', 'series', 'volume', 'watermarks',
'lastdetectedmodification', 'wikidataid', 'customprinturl']
Images columns: ['uuid', 'iiifurl', 'iiifthumburl', 'viewtype',
'sequence', 'width', 'height', 'maxpixels', 'openaccess', 'created',
'modified', 'depictstmsobjectid', 'assistivetext']
```

```python
# Correct join using objectid
merged = paintings.merge(
    images[["depictstmsobjectid", "iiifthumburl", "iiifurl"]],
    left_on="objectid",
    right_on="depictstmsobjectid",
    how="inner"
)
merged = merged.dropna(subset=["iiifthumburl"])

print(f"Paintings with images: {len(merged):,}")
print(f"\nSample image URL:")
print(merged["iiifthumburl"].iloc[0])
```

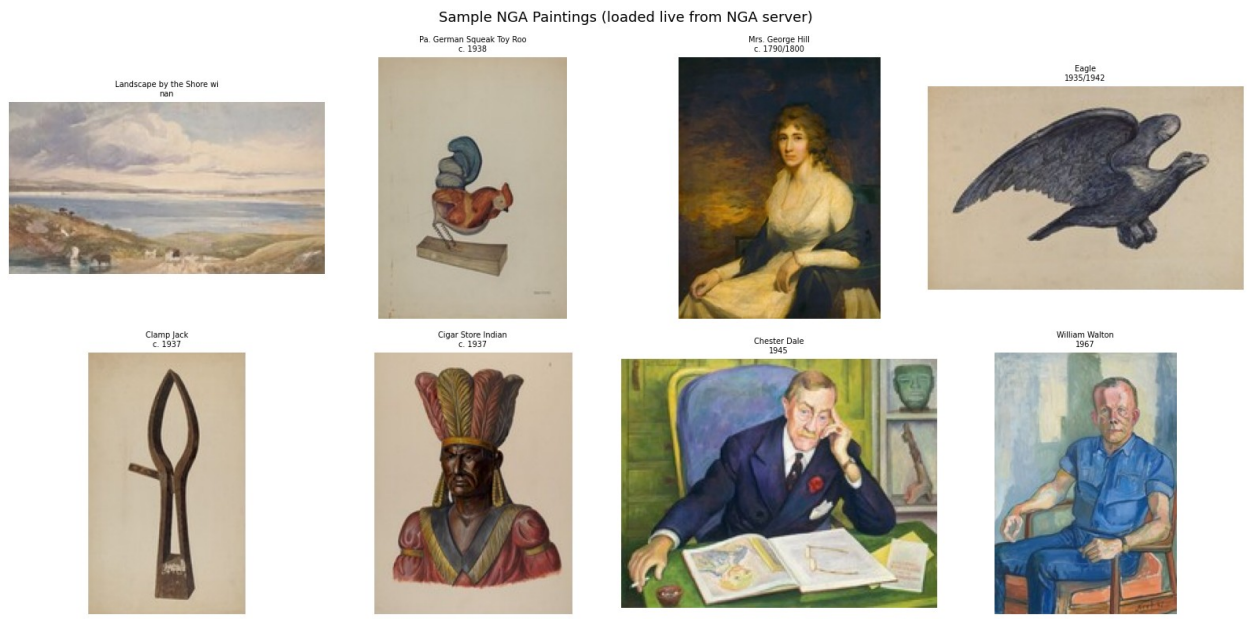
```
Paintings with images: 22,579

Sample image URL:
https://api.nga.gov/iiif/7b170a4c-9d44-475c-b294-cee6f43d88af/full/!
200,200/0/default.jpg
```

Data Preparation

Joined paintings with images using objectid — found 22,579 paintings with accessible images. Images are served via IIIF protocol meaning we can request any size on demand without downloading the full collection.

```python
import requests
from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt

def load_image(url):
    try:
        r = requests.get(url, timeout=10)
        return Image.open(BytesIO(r.content)).convert("RGB")
    except:
        return None

# Display 8  paintings
sample = merged.sample(8, random_state=42)

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
```

```
for ax, (_, row) in zip(axes.flat, sample.iterrows()):
    img = load_image(row["iiifthumburl"])
    if img:
        ax.imshow(img)
        ax.set_title(f"{str(row['title'])[:25]}\
n{str(row['displaydate'])}",
                     fontsize=7)
    ax.axis("off")

plt.suptitle("Sample NGA Paintings (loaded live from NGA server)",
fontsize=13)
plt.tight_layout()
plt.show()
```



several portraits visible (Mrs George Hill, Chester Dale, William Walton) a good similarity model should group these together despite different artists and time periods based on pose

```
from transformers import AutoImageProcessor, AutoModel
import torch
import torch.nn.functional as F

print(" DINOv2...")
processor = AutoImageProcessor.from_pretrained("facebook/dinov2-
small")
model     = AutoModel.from_pretrained("facebook/dinov2-small")
model.eval()
print(" DINOv2 loaded")
print(f"Embedding dimension: {model.config.hidden_size}")
```

```
 DINOv2...
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```

{"model_id":"8737e0ea373449a191ae3327d92db564","version_major":2,"version_minor":0}

```
The image processor of type `BitImageProcessor` is now loaded as a
fast processor by default, even if the model checkpoint was saved with
a slow processor. This is a breaking change and may produce slightly
different outputs. To continue using the slow processor, instantiate
this class with `use_fast=False`.
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"7ebfbdd93d864108a4769992e60e663f","version_major":2,"version_minor":0}

{"model_id":"742a255606eb44adaf6c446a4524485c","version_major":2,"version_minor":0}

{"model_id":"42aa8d7abef64b4fbb6ff4e094faf30f","version_major":2,"version_minor":0}

```
 DINOv2 loaded
Embedding dimension: 384
```

```python
import numpy as np

def get_embedding(image):
    """Get DINOv2 embedding for a single PIL image"""
    inputs  = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)
    # CLS token = summary of whole image
    embedding = outputs.last_hidden_state[:, 0, :]
    # L2 normalise so cosine similarity = dot product
    embedding = F.normalize(embedding, dim=-1)
    return embedding.squeeze().numpy()
```

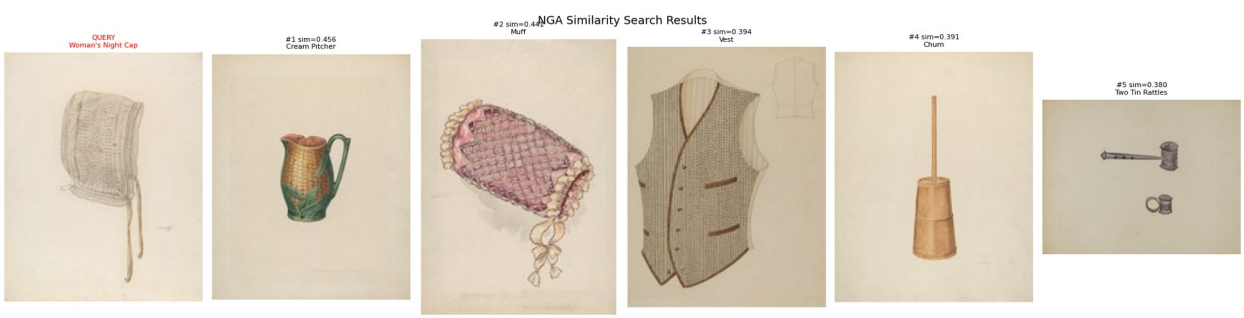
```python
# Load and embed 50 paintings
print("Loading and embedding 50 paintings...")
sample50 = merged.sample(50, random_state=99)

all_embeddings = []
all_metadata   = []

for _, row in sample50.iterrows():
    img = load_image(row["iiifthumburl"])
    if img:
        emb = get_embedding(img)
        all_embeddings.append(emb)
        all_metadata.append({
            "title":       str(row["title"])[:40],
            "date":        str(row["displaydate"]),
            "objectid":    str(row["objectid"]),
            "thumburl":    str(row["iiifthumburl"]),
        })

all_embeddings = np.array(all_embeddings)
print(f"Embedded {len(all_embeddings)} paintings")
print(f"Embedding matrix shape: {all_embeddings.shape}")
```

```
Loading and embedding 50 paintings...
Embedded 50 paintings
Embedding matrix shape: (50, 384)
```

```python
def find_similar(query_idx, embeddings, metadata, top_k=5):
    """Find top_k most similar paintings to the query"""
    query_emb = embeddings[query_idx]

    # Dot product = cosine similarity (vectors are normalised)
    similarities = embeddings @ query_emb

    # Sort by similarity, exclude the query itself
    ranked = np.argsort(-similarities)
    ranked = [i for i in ranked if i != query_idx][:top_k]

    return [(ranked[i], similarities[ranked[i]]) for i in
range(top_k)]

# Pick painting index 0 as query and find similar ones
query_idx = 0
similar   = find_similar(query_idx, all_embeddings, all_metadata)

print(f"Query painting: {all_metadata[query_idx]['title']}")
print(f"\nTop 5 most similar paintings:")
for idx, sim in similar:
    print(f"  [{sim:.3f}] {all_metadata[idx]['title']}")
```

```
Query painting: Woman's Night Cap

Top 5 most similar paintings:
  [0.456] Cream Pitcher
  [0.441] Muff
  [0.394] Vest
  [0.391] Churn
  [0.380] Two Tin Rattles
```

```python
fig, axes = plt.subplots(1, 6, figsize=(20, 5))

# Show query painting
query_img = load_image(all_metadata[query_idx]["thumburl"])
axes[0].imshow(query_img)
axes[0].set_title(f"QUERY\n{all_metadata[query_idx]['title']}",
                    fontsize=8, color="red")
axes[0].axis("off")

# Show similar paintings
for i, (idx, sim) in enumerate(similar):
    img = load_image(all_metadata[idx]["thumburl"])
    if img:
        axes[i+1].imshow(img)
        axes[i+1].set_title(f"#{i+1} sim={sim:.3f}\n{all_metadata[idx]
['title']}",
                            fontsize=8)
    axes[i+1].axis("off")

plt.suptitle("NGA Similarity Search Results", fontsize=13)
plt.tight_layout()
plt.show()
```



##Observations

###Query painting: Woman's Night Cap

model retreaved 5 similar paintings and as we can see they are similar in:

1. Background cream color and drawing atyle
2. Night cap, cream pitcher, muff, vest, churn and tin rattles are all from the same NGA collection of americal folk/ art illustrations

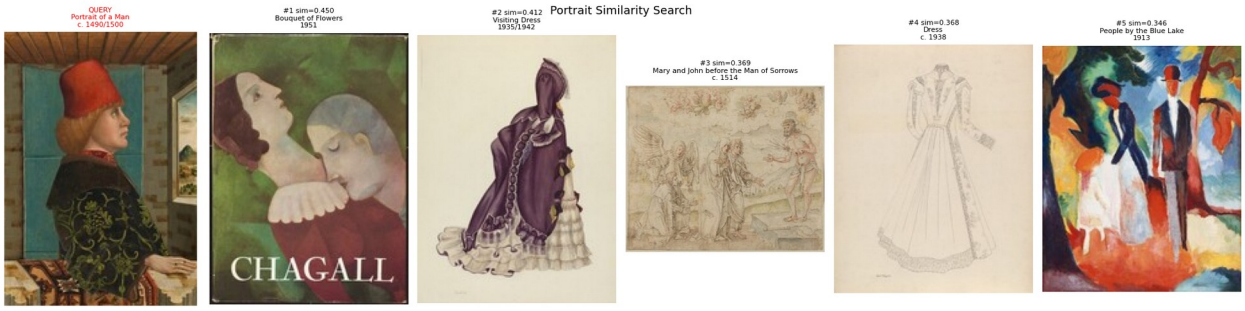
3. First 3 share the similar pattern as well, the knit pattern or checkered pattern
4. last two make less sense but the drawing style and bg goes well

DINOV2 found similarities without any labels purely from visual features

```
#portrait
for i, meta in enumerate(all_metadata):
    if any(word in meta["title"].lower() for word in
            ["portrait", "mrs", "mr", "general", "colonel", "lady",
"sir"]):
        print(f"Index {i}: {meta['title']}")
```

```
Index 26: Portrait of a Man
```

```
query_idx = 26
similar   = find_similar(query_idx, all_embeddings, all_metadata)

fig, axes = plt.subplots(1, 6, figsize=(20, 5))

query_img = load_image(all_metadata[query_idx]["thumburl"])
axes[0].imshow(query_img)
axes[0].set_title(f"QUERY\n{all_metadata[query_idx]['title']}\
n{all_metadata[query_idx]['date']}",
                    fontsize=8, color="red")
axes[0].axis("off")

for i, (idx, sim) in enumerate(similar):
    img = load_image(all_metadata[idx]["thumburl"])
    if img:
        axes[i+1].imshow(img)
        axes[i+1].set_title(
            f"#{i+1} sim={sim:.3f}\n{all_metadata[idx]['title']}\
n{all_metadata[idx]['date']}",
            fontsize=8)
    axes[i+1].axis("off")

plt.suptitle("Portrait Similarity Search", fontsize=13)
plt.tight_layout()
plt.show()

print(f"\nQuery: {all_metadata[query_idx]['title']}")
print(f"\nTop 5 similar:")
for idx, sim in similar:
    print(f"  [{sim:.3f}] {all_metadata[idx]['title']}
({all_metadata[idx]['date']})")
```




```
Query: Portrait of a Man

Top 5 similar:
  [0.450] Bouquet of Flowers (1951)
  [0.412] Visiting Dress (1935/1942)
  [0.369] Mary and John before the Man of Sorrows (c. 1514)
  [0.368] Dress (c. 1938)
  [0.346] People by the Blue Lake (1913)
```

###observation

Well, the paintings dont seem all that similar as can be seen from the low similarity scores meaning model itself isnt very confident on the similarity part

How to fix: Add more paintings

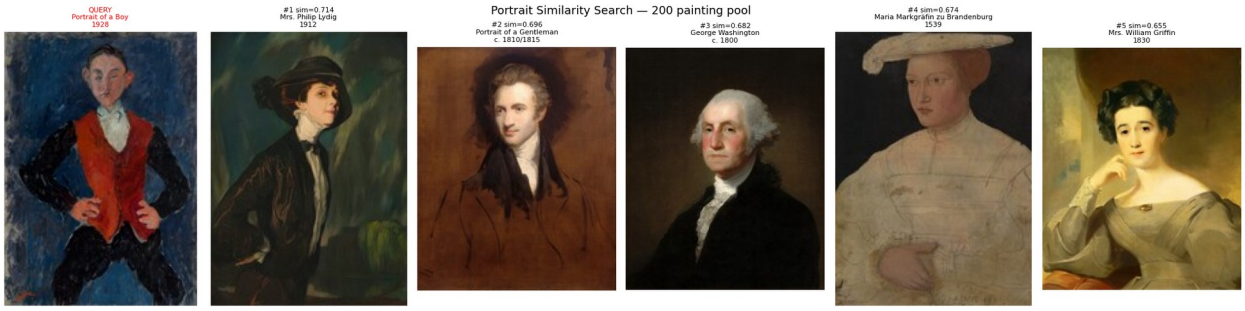
```
print("Embedding 200 paintings...")
sample200 = merged.sample(200, random_state=42)

all_embeddings2 = []
all_metadata2   = []

for _, row in sample200.iterrows():
    img = load_image(row["iiifthumburl"])
    if img:
        emb = get_embedding(img)
        all_embeddings2.append(emb)
        all_metadata2.append({
            "title":    str(row["title"])[:40],
            "date":     str(row["displaydate"]),
            "thumburl": str(row["iiifthumburl"]),
        })

all_embeddings2 = np.array(all_embeddings2)
print(f"Done. Embedded {len(all_embeddings2)} paintings")
```

```
Embedding 200 paintings...
Done. Embedded 199 paintings
```

```
# Find portraits in the bigger sample
for i, meta in enumerate(all_metadata2):
    if any(word in meta["title"].lower() for word in
           ["portrait", "mrs", "mr", "general", "colonel", "lady",
"sir"]):
        print(f"Index {i}: {meta['title']}")
```

```
Index 2: Mrs. George Hill
Index 9: Portrait of a Boy
Index 16: Family Portrait
Index 29: Portrait of a Gentleman
Index 49: Mrs. Philip Lydig
Index 120: Mrs. William Griffin
```

```
query_idx2 = 9
similar2   = find_similar(query_idx2, all_embeddings2, all_metadata2)

fig, axes = plt.subplots(1, 6, figsize=(20, 5))

query_img = load_image(all_metadata2[query_idx2]["thumburl"])
axes[0].imshow(query_img)
axes[0].set_title(f"QUERY\n{all_metadata2[query_idx2]['title']}\
n{all_metadata2[query_idx2]['date']}",
                  fontsize=8, color="red")
axes[0].axis("off")

for i, (idx, sim) in enumerate(similar2):
    img = load_image(all_metadata2[idx]["thumburl"])
    if img:
        axes[i+1].imshow(img)
        axes[i+1].set_title(
            f"#{i+1} sim={sim:.3f}\n{all_metadata2[idx]['title']}\
n{all_metadata2[idx]['date']}",
            fontsize=8)
    axes[i+1].axis("off")

plt.suptitle("Portrait Similarity Search — 200 painting pool",
fontsize=13)
plt.tight_layout()
plt.show()

print(f"Query: {all_metadata2[query_idx2]['title']}")
print(f"\nTop 5 similar:")
for idx, sim in similar2:
    print(f"  [{sim:.3f}] {all_metadata2[idx]['title']}
({all_metadata2[idx]['date']})")
```




```
Query: Portrait of a Boy

Top 5 similar:
  [0.714] Mrs. Philip Lydig (1912)
  [0.696] Portrait of a Gentleman (c. 1810/1815)
  [0.682] George Washington (c. 1800)
  [0.674] Maria Markgräfin zu Brandenburg (1539)
  [0.655] Mrs. William Griffin (1830)
```

Now thats what we're talkin about!!

All are portraits similarity scored jumped with more paintings

human figures as main subject, query and 1st painting even share the same pose, faces focal point, same bg

aka dinov2 understands portraitr similarity w/o training

pool size matters

```
import numpy as np

# Compute similarity matrix for all 200 paintings
sim_matrix = all_embeddings2 @ all_embeddings2.T

# Find portrait indices
portrait_indices = [i for i, meta in enumerate(all_metadata2)
                   if any(word in meta["title"].lower() for word in
                          ["portrait", "mrs", "mr", "general",
                           "colonel", "lady", "sir"])]

print(f"Portrait paintings in pool: {len(portrait_indices)}")
print(f"Portrait titles:")
for i in portrait_indices:
    print(f"  [{i}] {all_metadata2[i]['title']}")

# Precision@K for portraits
def precision_at_k(query_idx, relevant_indices, embeddings, k=5):
    sims = embeddings @ embeddings[query_idx]
    sims[query_idx] = -1
```

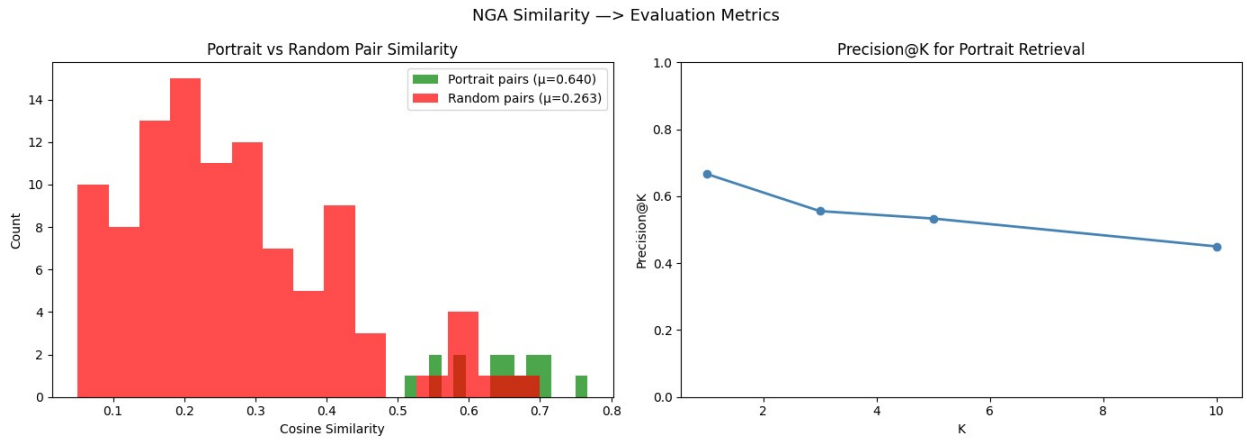
```
    top_k = np.argsort(-sims)[:k]
    relevant_in_top_k = sum(1 for i in top_k if i in relevant_indices)
    return relevant_in_top_k / k

# Compute Precision@5 for each portrait query
print(f"\n=== PRECISION@5 FOR PORTRAIT RETRIEVAL ===")
precisions = []
for qi in portrait_indices:
    other_portraits = [i for i in portrait_indices if i != qi]
    p = precision_at_k(qi, other_portraits, all_embeddings2, k=5)
    precisions.append(p)
    print(f"  {all_metadata2[qi]['title']:<35} P@5: {p:.2f}")

print(f"\n  Mean Precision@5: {np.mean(precisions):.3f}")
```

```
Portrait paintings in pool: 6
Portrait titles:
  [2] Mrs. George Hill
  [9] Portrait of a Boy
  [16] Family Portrait
  [29] Portrait of a Gentleman
  [49] Mrs. Philip Lydig
  [120] Mrs. William Griffin

=== PRECISION@5 FOR PORTRAIT RETRIEVAL ===
  Mrs. George Hill                    P@5: 0.40
  Portrait of a Boy                   P@5: 0.60
  Family Portrait                     P@5: 0.60
  Portrait of a Gentleman             P@5: 0.60
  Mrs. Philip Lydig                   P@5: 0.60
  Mrs. William Griffin                P@5: 0.40

  Mean Precision@5: 0.533
```

precision@5 of 0.533 means out of every 5 paintings retrieved for a portrait query, 2-3 are also portraits and thats without any training at all

```
# Cosine similarity between portrait pairs vs random pairs
portrait_sims = []
for i in range(len(portrait_indices)):
    for j in range(i+1, len(portrait_indices)):
        portrait_sims.append(sim_matrix[portrait_indices[i],
portrait_indices[j]])

random_sims = []
non_portraits = [i for i in range(len(all_metadata2)) if i not in
portrait_indices]
for _ in range(100):
    i = np.random.choice(portrait_indices)
    j = np.random.choice(non_portraits)
```

```
    random_sims.append(sim_matrix[i, j])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Similarity distributions
axes[0].hist(portrait_sims, bins=15, alpha=0.7, color="green",
             label=f"Portrait pairs (μ={np.mean(portrait_sims):.3f})")
axes[0].hist(random_sims,   bins=15, alpha=0.7, color="red",
             label=f"Random pairs (μ={np.mean(random_sims):.3f})")
axes[0].set_xlabel("Cosine Similarity")
axes[0].set_ylabel("Count")
axes[0].set_title("Portrait vs Random Pair Similarity")
axes[0].legend()

# Plot 2: Precision@K for different K values
k_values  = [1, 3, 5, 10]
p_at_k_vals = []
for k in k_values:
    ps = []
    for qi in portrait_indices:
        other = [i for i in portrait_indices if i != qi]
        ps.append(precision_at_k(qi, other, all_embeddings2, k=k))
    p_at_k_vals.append(np.mean(ps))

axes[1].plot(k_values, p_at_k_vals, "o-", color="steelblue",
linewidth=2)
axes[1].set_xlabel("K")
axes[1].set_ylabel("Precision@K")
axes[1].set_title("Precision@K for Portrait Retrieval")
axes[1].set_ylim(0, 1)

plt.suptitle("NGA Similarity -> Evaluation Metrics", fontsize=13)
plt.tight_layout()
plt.savefig("metrics.png")
plt.show()

print(f"\n EVALUATION SUMMARY")
print(f"Portrait pair similarity:  {np.mean(portrait_sims):.3f}")
print(f"Random pair similarity:    {np.mean(random_sims):.3f}")
print(f"Similarity gap:            {np.mean(portrait_sims) -
np.mean(random_sims):.3f}")
print(f"Mean Precision@5:          {np.mean(precisions):.3f}")
```




```
 EVALUATION SUMMARY
Portrait pair similarity:  0.640
Random pair similarity:    0.263
Similarity gap:            0.377
Mean Precision@5:          0.533
```

##Results

portrait pairs 0.640 vs random pairs: 0.263 they have a gap of 0.377 i.e portraits score 2.4x higher similarity with each other than with random paintings. This embedding space has separted portriats from other artwrok types with 0 training

precision@k chart starting from 0.67 (k=1) stays above 0.5 even at k=10 which means that even if we ask for 10 results, 50%+ are portraits

No accuracy metric used because there are no ground truth labels this is unsupervised similarity, not classification